

UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO DE FIN DE GRADO

GRADO EN INGENIERÍA DEL SOFTWARE



# APLICACIÓN WEB PARA LA GESTIÓN DE HÁBITOS DIARIOS

**AUTOR:** MYKHAYLO STANISLAY KULYS KHUDZEY

**TUTOR:** BORJA BORDEL SANCHEZ

Madrid, 2025



## AGRADECIMIENTOS

En primer lugar, me gustaría agradecerle a mi tutor Borja Bordel Sánchez por su apoyo, paciencia y orientación durante el desarrollo de este proyecto.

Y principalmente quería agradecer a mi familia por su apoyo durante la carrera, en especial a mi madre que siempre ha estado allí cuando la he necesitado y por la paciencia que ha tenido conmigo en este largo camino.

# RESUMEN

En la actualidad, numerosas personas buscan formas de incrementar su rendimiento personal y crear una rutina constante que promueva su bienestar. Sin embargo, la falta de instrumentos simples para manejar hábitos puede dificultar la creación de una disciplina estable.

Este trabajo final de grado aborda el desarrollo de una plataforma en línea enfocada en la administración de hábitos. La herramienta facilita a los usuarios la adición, alteración, supresión y finalización de costumbres reiterativas de forma intuitiva, promoviendo la organización y el seguimiento de objetivos personales.

El diseño de la plataforma ha empleado Angular para la parte frontal, Node.js utilizando Express para el entorno servidor y PostgreSQL para la administración de información. Incorpora funciones como autenticación, monitoreo de costumbres, ajustes y una interfaz sencilla para usar.

A lo largo del documento se detallan los requisitos funcionales y técnicos, el diseño del sistema, las tecnologías empleadas, el proceso de desarrollo, las pruebas realizadas y futuras líneas de mejora.

# ABSTRACT

Nowadays, many people are looking for ways to improve their personal performance and create a consistent routine that promotes their well-being. However, the lack of simple tools for managing habits can make it difficult to establish a stable discipline.

This final degree project addresses the development of an online platform focused on habit management. The tool makes it easy for users to add, modify, delete, and terminate repetitive habits intuitively, promoting the organization and tracking of personal goals.

The platform was designed using Angular for the front-end, Node.js using Express for the server environment, and PostgreSQL for data management. It incorporates features such as authentication, habit monitoring, settings, and a user-friendly interface.

The document details the functional and technical requirements, system design, technologies used, the development process, testing, and future lines of improvement.

# Índice

|                                                                     |    |
|---------------------------------------------------------------------|----|
| AGRADECIMIENTOS.....                                                | 2  |
| RESUMEN.....                                                        | 3  |
| ABSTRACT .....                                                      | 4  |
| 1.Introducción.....                                                 | 7  |
| 1.1.    Motivación .....                                            | 7  |
| 1.2 Objetivos.....                                                  | 8  |
| 2.    Estado del arte.....                                          | 9  |
| 2.1    Aplicaciones actuales de gestión de hábitos .....            | 9  |
| 2.2    Comparativa general.....                                     | 12 |
| 2.3    Tecnologías y lenguajes utilizados en el desarrollo web..... | 13 |
| 2.4    Herramientas utilizadas en el desarrollo.....                | 15 |
| 3    Análisis y requisitos .....                                    | 17 |
| 3.1    Requisitos funcionales .....                                 | 17 |
| 3.2    Requisitos no funcionales .....                              | 19 |
| 3.3    Roles de usuario .....                                       | 20 |
| 3.4    Casos de uso .....                                           | 20 |
| 3.4.1 Casos de uso del usuario .....                                | 21 |
| 3.4.2 Casos de uso del Administrador.....                           | 22 |
| 4    Diseño del sistema.....                                        | 23 |
| 4.1    Arquitectura general .....                                   | 23 |
| 4.2    Diagrama de arquitectura.....                                | 24 |
| 4.3 Diseño de la base de datos .....                                | 25 |
| 4.3.1 Entidades.....                                                | 25 |
| 4.3.2 Diagrama de la base de datos .....                            | 27 |
| 4.4 Diseño del backend .....                                        | 27 |

|       |                                       |    |
|-------|---------------------------------------|----|
| 4.4.1 | Rutas .....                           | 28 |
| 4.4.2 | Middlewares .....                     | 34 |
| 4.4.3 | Auth.controller.js – Controlador..... | 35 |
| 4.4.4 | Modelo de datos.....                  | 36 |
| 4.4.5 | Seguridad.....                        | 36 |
| 4.5   | Diseño del frontend.....              | 37 |
| 4.5.1 | Estructura general del frontend ..... | 37 |
|       | Componentes .....                     | 37 |
| 5     | Pruebas del sistema .....             | 51 |
| 5.1.  | Pruebas unitarias .....               | 51 |
| 5.2   | Pruebas funcionales.....              | 51 |
| 5.3.  | Pruebas de API con Postman .....      | 52 |
| 6.    | Impacto social y medioambiental ..... | 54 |
| 6.1   | Impacto social .....                  | 54 |
| 6.2   | Impacto medioambiental .....          | 55 |
| 7.    | Conclusiones .....                    | 56 |
| 7.1   | Líneas futuras de mejora.....         | 56 |
|       | Bibliografía.....                     | 59 |

# 1.Introducción

En mi experiencia, vivimos en un mundo cada vez más abarrotado de distracciones que complican la concentración y afectan negativamente la productividad diaria de las personas. Cada vez más personas buscan mejorar salud mental y productividad estableciendo rutinas saludables y hábitos.

Establecer una disciplina es fundamental para poder alcanzar las metas personales y mantener un bienestar mental y físico. Sin embargo, el principal reto no solo prevalece en definir correctamente estos hábitos, sino también en realizar un seguimiento constante que permita consolidarlos a lo largo del tiempo estableciendo una disciplina y poder visualizar los logros obtenidos.

Con el auge del desarrollo personal, surge la idea de presentar el desarrollo de esta aplicación web sencilla e intuitiva que permita a los usuarios gestionar sus hábitos y ver su progreso a lo largo del tiempo. Esta herramienta intenta servir como apoyo en la construcción de una rutina personalizada, promoviendo la constancia y la motivación a través de una interfaz accesible.

**El desarrollo del proyecto fue controlado mediante el sistema de versiones Git y se encuentra disponible en el repositorio:**

<https://github.com/mkhudzey/HABITIVITY.git>

## 1.1. Motivación

Durante la última década, el crecimiento de las redes sociales y, especialmente, el surgimiento de aplicaciones basadas en el "scroll infinito" han tenido un impacto significativo en los estilos de vida de millones de individuos. Estas plataformas, creadas para atraer de manera constante el interés del usuario, han ocasionado una creciente disminución de concentración, disciplina y tiempo personal.

Desde mi perspectiva personal, alcancé un punto en el que comprendí que mi grado de rendimiento y bienestar no eran los apropiados para un individuo de mi edad. Tras esta



reflexión, opté por implementar modificaciones importantes en mi rutina diaria con la finalidad de elevar mi calidad de vida.

El primer paso consistió en tratar de integrar nuevos hábitos en mi rutina diaria. Sin embargo, pronto me percaté de que la ausencia de seguimiento y constancia provocaba que olvidase realizarlos. Esta dificultad me impulsó a crear una solución práctica que me facilitara registrar, observar y monitorear estos hábitos.

Por lo tanto, surgió el concepto de desarrollar una aplicación web que simplificara la supervisión y seguimiento de hábitos, junto con una interfaz visual que permita visualizar el avance de estos. Esta herramienta surge de un requerimiento personal que podría ser de anhelo para cualquier individuo que desee construir su rutina.

## 1.2 Objetivos

El objetivo principal de este trabajo de fin de grado es el desarrollo de una aplicación web intuitiva y accesible que permita a los usuarios gestionar sus hábitos personales, haciendo posible su seguimiento y fomentando la constancia en la construcción de rutinas organizadas. La aplicación está diseñada para adaptarse a diferentes tipos de hábitos, incluyendo hábitos diarios, semanales y mensuales.

Este objetivo se divide en los siguientes objetivos específicos:

- Diseñar e implementar una interfaz de usuario sencilla e intuitiva que facilite la interacción.
- Permitir a los usuarios registrar, editar, eliminar y completar hábitos, adaptándose a diferentes necesidades y tipos de rutinas.
- Incorporar un sistema de autenticación para que cada usuario pueda acceder de forma segura.
- Ofrecer una representación visual del progreso de los hábitos que motive al usuario.
- Desarrollar una arquitectura cliente-servidor, utilizando Angular, Node.js y PostgreSQL.
- Garantizar la persistencia de datos y la correcta comunicación entre frontend y backend.

- Restringir el acceso a determinadas funcionalidades mediante control de roles.
- Garantizar la autenticación segura mediante JWT y almacenamiento cifrado de contraseñas.

## 2. Estado del arte

El desarrollo de hábitos personales ha conseguido una gran relevancia en estos últimos años a causa del interés en el bienestar físico, mental y la mejora de la productividad. Esta tendencia se ve reflejada en la aparición de diversas aplicaciones móviles y plataformas web centradas en el seguimiento de rutinas, objetivos personales y visualización del progreso.

Existen varias herramientas actualmente, la mayoría se presentan como aplicaciones móviles, lo que puede ser una dificultad para ciertos usuarios. Además, varias de estas aplicaciones presentan limitaciones en frecuencia de seguimiento o claridad visual. Este TFG ofrece una aplicación web accesible desde cualquier navegador, sin necesidad de instalación, y que permita un seguimiento personalizado diario, semanal y mensual supone una mejora frente a otras propuestas.

### 2.1 Aplicaciones actuales de gestión de hábitos

- **Habitica**

Habitica es una aplicación de gestión de hábitos con una propuesta curiosa. Convertir el proceso de creación de hábitos y tareas en un juego de rol. Cada hábito o tarea cumplida otorga puntos de experiencia y recompensas que permiten mejorar un personaje. Está disponible en versión web y móvil.

- **Ventajas:** Altamente motivadora para usuarios que disfrutan del uso de videojuegos parecidos. Permite crear hábitos y tareas con diferentes periodicidades.
- **Inconvenientes:** El enfoque de juego puede parecer excesivamente un videojuego, lo que puede provocar que se pierda de vista el objetivo general y ser poco centrado en la productividad para ciertos perfiles.

Además, en opinión del autor, la interfaz puede resultar compleja o recargada, lo que disipa el objetivo principal de la aplicación: el desarrollo disciplinado de hábitos.

- **Fabulous**

Fabulous es una aplicación móvil diseñada especialmente para establecer rutinas para el bienestar físico y mental. Utiliza programas guiados, recordatorios diarios y un diseño atractivo para motivar la adopción de hábitos saludables.

- **Ventajas:** Muy visual e intuitiva para usuarios que buscan mejorar su salud y bienestar general. Ideal para usuarios novatos que necesitan que los guíen.
- **Inconvenientes:** La aplicación se enfoca principalmente en la salud física y el desarrollo de rutinas predefinidas, lo que ofrece poca flexibilidad para crear hábitos personalizados o con diferentes frecuencias. Además, su interfaz puede resultar algo compleja para realizar un seguimiento detallado de hábitos y su progreso, especialmente si se desean estadísticas o representaciones visuales.

- **HabitBull**

HabitBull permite registrar hábitos y hacer seguimiento de forma diaria o semanal. Tiene un enfoque más profesional y proporciona estadísticas, calendarios de cumplimiento y recordatorios.

- **Ventajas:** Permite cierta personalización de hábitos, e incluye gráficas de progreso básicas. Útil para usuarios que buscan simplicidad.
- **Inconvenientes:** La interfaz puede resultar al usuario poco intuitiva lo que hace complicado el uso de la aplicación y no soporta hábitos mensuales. Tampoco es especialmente visual.

- **Google Tasks**

Aunque no es una aplicación diseñada para los hábitos, Google Tasks permite organizar tareas de forma simple, integrándose con Gmail y Google Calendar.

Algunos usuarios la usan para crear hábitos manualmente como tareas recurrentes.

- **Ventajas:** Integración con otros productos de Google. Accesible tanto desde el navegador como desde la app móvil.
- **Inconvenientes:** No está diseñada para seguimiento de hábitos. No cuenta con estadísticas, visualización de progreso.

- **Habitify**

Habitify es una aplicación de gestión de hábitos disponible tanto en dispositivos móviles como en navegador web. Se caracteriza por una interfaz moderna y limpia, centrada en la productividad personal y el seguimiento detallado.

- **Ventajas:** Su interfaz es clara y intuitiva para el usuario. Permite crear hábitos diarios, semanales y con recordatorios personalizados. Incluye estadísticas y resúmenes del progreso, y sincroniza entre múltiples dispositivos.
- **Inconvenientes:** Algunas funcionalidades están limitadas a la versión de pago. Además, aunque incluye estadísticas, la representación gráfica no es tan diversa como en otras soluciones.

- **Esta aplicación**

Esta aplicación web de gestión de hábitos ha sido diseñada específicamente para ofrecer una solución clara, flexible y accesible. A diferencia de las anteriores, no requiere instalación y permite crear hábitos diarios, semanales y mensuales. Incluye visualización del progreso con gráficos interactivos.

- **Ventajas:** Accesible desde navegador, sin instalación. Soporta múltiples frecuencias de hábito. Incluye visualizaciones como histograma, gráfica tipo donut y línea temporal. Interfaz limpia y centrada en la productividad.
- **Inconvenientes:** Al ser una aplicación web sencilla, no incluye notificaciones push ni sistema de recompensas o insignias.

## 2.2 Comparativa general

A partir del análisis de las aplicaciones actuales de gestión de hábitos, se pueden extraer las siguientes conclusiones comparativas respecto a la aplicación desarrollada:

- **Flexibilidad**

Aplicaciones como Habitica, HabitBull y Habitify son personalizables respecto en la creación de hábitos, pero solo mi aplicación permite una configuración clara y diferenciada para hábitos diarios, semanales y mensuales, lo cual proporciona una mayor separación en el control de la rutina personal.

- **Visualización del progreso**

Cada herramienta aborda la visualización de manera distinta. Habitica emplea metáforas visuales propias de videojuegos, Fabulous hace uso de imágenes motivacionales y programas guiados, y HabitBull muestra calendarios sencillos. Habitify ofrece al usuario estadísticas básicas pero limitadas en su versión gratuita. Por su parte, mi aplicación destaca por ofrecer gráficas interactivas y específicas para cada tipo de hábito, como histogramas, gráficos de anillo y líneas temporales, facilitando el seguimiento visual.

- **Accesibilidad**

Mientras que aplicaciones como Fabulous, HabitBull o Habitify requieren instalación en dispositivos móviles, mi aplicación puede utilizarse directamente desde cualquier navegador web, sin necesidad de instalación ni consumo de recursos de memoria en el dispositivo del usuario.

- **Objetivo**

Aplicaciones como Habitica y Fabulous priorizan la motivación emocional o el sistema de recompensa a través de insignias o objetos en la aplicación como principal incentivo. En cambio, mi aplicación se enfoca en una experiencia funcional y enfocada en el progreso cuantificable, con una interfaz limpia, sin

distracciones enfocada en el objetivo principal, la consolidación de los hábitos.

| Aplicación      | Flexibilidad                                | Visualización                                       | Accesibilidad        | Enfoque                                      |
|-----------------|---------------------------------------------|-----------------------------------------------------|----------------------|----------------------------------------------|
| Habitica        | Solo diaria, con personalización básica     | Elementos de juego tipo RPG                         | Requiere instalación | Sistema de recompensas parecidas a un juego  |
| Fabulous        | Limitada a rutinas predefinidas             | Imágenes motivacionales y secuencias guiadas        | Requiere instalación | Salud y bienestar con enfoque emocional      |
| HabitBull       | Diaria y semanal con cierta personalización | Calendarios simples                                 | Requiere instalación | Seguimiento técnico                          |
| Google Tasks    | Muy limitada (solo tareas)                  | No aplica (sin seguimiento visual)                  | Accesible desde web  | Gestión de tareas, no hábitos                |
| Habitify        | Diaria / Semanal, buena interfaz            | Estadísticas básicas, limitadas en versión gratuita | Requiere instalación | Productividad con diseño minimalista         |
| Esta Aplicación | Completa: diaria, semanal y mensual         | Gráficas interactivas (donut, histograma, línea)    | Web, sin instalación | Seguimiento claro, centrado en progreso real |

## 2.3 Tecnologías y lenguajes utilizados en el desarrollo web

La aplicación web desarrollada en este proyecto hace uso de varias tecnologías actuales de JavaScript y en lenguajes utilizados en el ámbito del desarrollo web. A continuación, se describen los principales lenguajes y frameworks empleados.

### Lenguajes de programación

- TypeScript

Es el lenguaje que utilicé en el frontend, ya que Angular lo requiere. Me resultó muy útil por su tipado estático y por permitir mayor control al definir interfaces y estructuras, lo que ayudó a evitar errores comunes.

- **JavaScript**

Usé JavaScript en el backend con Node.js. Me permitió trabajar de forma fluida con las peticiones del frontend, ya que podía manejar operaciones como acceder a la base de datos sin que el servidor se quedara esperando o se bloqueara.

- **HTML5**

Me sirvió para estructurar la interfaz del usuario. Definí la organización de los elementos visuales desde aquí.

- **CSS3**

Usado para aplicar estilos visuales a los componentes del frontend para una interfaz más clara e intuitiva.

- **SQL**

Usé este lenguaje para trabajar con PostgreSQL. Lo empleé para crear las tablas, establecer relaciones entre ellas y realizar las consultas necesarias.

## **Frameworks y tecnologías**

### **Angular**

Es el framework que usé para construir el frontend. Me permitió trabajar de forma modular con componentes y organizar mejor la aplicación. También aproveché su sistema de servicios y rutas para gestionar vistas y datos.

### **Node.js**

En el backend, Node.js me permitió escribir JavaScript del lado del servidor. Funciona muy bien para aplicaciones ligeras y rápidas, y me gustó su forma de manejar múltiples peticiones a la vez.

### **Express.js**

Sobre Node.js utilicé Express, que facilita mucho la creación de rutas, middlewares y controladores. Con esto pude gestionar las peticiones del frontend de forma clara y organizada.

### **PostgreSQL**

Elegí este sistema de bases de datos por su estabilidad y porque permite manejar relaciones complejas. Lo usé para almacenar toda la información de usuarios, hábitos y logs de manera estructurada.

Este conjunto de tecnologías proporciona una arquitectura basada en el modelo cliente-servidor, que permite un desarrollo organizado, mantenible y adaptable a futuros requisitos funcionales.

## **2.4 Herramientas utilizadas en el desarrollo**

Durante el desarrollo de la aplicación web para la gestión de hábitos se han empleado diversas herramientas de desarrollo, prueba y gestión que permitieron estructurar el trabajo de forma eficiente y garantizar la funcionalidad de la aplicación final. A continuación, se detallan las herramientas más relevantes utilizadas en cada fase del proyecto:

### **Visual Studio Code**

Fue el editor principal para todo el desarrollo del proyecto, tanto en el frontend como en el backend. Su interfaz intuitiva y con su soporte para múltiples lenguajes lo convirtieron en una opción ideal.

### **Postman**

Postman fue fundamental en la fase de desarrollo del backend, permitiendo realizar pruebas de las APIs REST creadas con Node.js y Express. Se empleó para simular peticiones HTTP (GET, POST, PUT, DELETE) y comprobar el funcionamiento de la llamadas desde el backend, además de probar el flujo de autenticación con tokens y crear colecciones organizadas por funcionalidades



(hábitos, usuarios, login, etc.). También se utilizó para documentar el comportamiento esperado de los endpoints.

### **Nodemon**

Durante el desarrollo del backend, se utilizó Nodemon, una herramienta que permite reiniciar automáticamente el servidor Node.js cada vez que se detectan cambios en los archivos. Esto permitió acelerar el ciclo de desarrollo y reducir errores por olvidos de reinicio.

### **PgAdmin**

Para la administración de la base de datos PostgreSQL, se utilizó PgAdmin, una herramienta gráfica que facilitó la visualización de tablas, ejecución de consultas SQL, definición de relaciones entre entidades y comprobación de la persistencia de datos. También fue útil para verificar la información tras realizar operaciones desde la interfaz web.

### **Angular CLI y Node Package Manager**

Se utilizó Angular CLI para la generación de componentes, servicios y rutas de forma automatizada y se usó npm para la instalación y gestión de dependencias del proyecto frontend, incluyendo librerías de formularios, validaciones, JWT y componentes visuales.

## 3 Análisis y requisitos

En esta sección se especifican los requisitos funcionales y no funcionales de la aplicación proporcionando una visión general de su funcionamiento desde el punto de vista del usuario. Esta fase es clave para asegurar que el producto desarrollado cumple con las necesidades planteadas.

### 3.1 Requisitos funcionales

Los requisitos funcionales describen las acciones específicas que la aplicación debe ser capaz de realizar. En este caso, esta aplicación web de gestión de hábitos, los principales son:

- **RF-1. Registro de usuarios:** El usuario podrá crear una cuenta introduciendo un correo electrónico y una contraseña.
- **RF-2. Autenticación:** El usuario podrá acceder a su cuenta introduciendo su nombre de usuario y contraseña.
- **RF-3. Gestión de hábitos:**
  - **RF-3.1:** El usuario podrá crear hábitos diarios proporcionando un nombre y una cantidad valida.
  - **RF-3.2:** El usuario podrá crear hábitos semanales proporcionando un nombre y una cantidad valida.
  - **RF-3.3:** El usuario podrá crear hábitos mensuales proporcionando un nombre y una cantidad valida.
  - **RF-3.4:** El usuario podrá modificar un hábito diario que haya creado el.
  - **RF-3.5:** El usuario podrá modificar un hábito semanal que haya creado el.
  - **RF-3.6:** El usuario podrá modificar un hábito mensual que haya creado el.
  - **RF-3.7:** El usuario podrá eliminar un hábito diario que haya creado el.

- **RF-3.8:** El usuario podrá eliminar un hábito semanal que haya creado el.
- **RF-3.9:** El usuario podrá eliminar un hábito mensual que haya creado el.
- **RF-3.10:** El usuario podrá marcar como hecho un hábito diario que haya creado este.
- **RF-3.11:** El usuario podrá marcar como hecho un hábito semanal que haya creado este.
- **RF-3.12:** El usuario podrá marcar como hecho un hábito mensual que haya creado este.
- **RF-4. Visualización del progreso:**
  - **RF-4.1:** El usuario podrá visualizar su progreso de hábitos diarios en función del rango de días que seleccione (semana, quincena y mes).
  - **RF-4.2:** El usuario podrá visualizar el porcentaje de hábitos diarios completados en función del rango de días que seleccione.
  - **RF-4.3:** El usuario podrá visualizar el progreso de sus hábitos semanales de forma grafica
  - **RF-4.4:** El usuario podrá visualizar el progreso de los últimos 5 meses de sus hábitos mensuales.
- **RF-5. Visualización de perfil:** El usuario deberá ser capaz de visualizar los datos de su cuenta.
- **RF-6. Gestión de usuario:**
  - **RF-6.1:** El sistema debe terminar completamente la sesión del usuario al hacer clic en el botón de Cerrar sesión.
  - **RF-6.2:** El usuario deberá ser capaz de cambiar la contraseña de su cuenta al hacer clic a Cambiar contraseña y proporcionando la información requerida.
  - **RF-6.3 :** El sistema debe eliminar la cuenta del usuario al hacer clic en el botón de Eliminar cuenta.
- **RF-7. Visualización de logros:** El usuario podrá visualizar la cantidad de hábitos de cada tipo ha completado en total.

- **RF-8. Panel de Administrador:** Un usuario con rol de “admin” podrá acceder al panel de administrador
  - **RF-8.1. Visualización de estadísticas:** El administrador podrá visualizar las estadísticas de la aplicación como la cantidad usuarios, hábitos diarios, semanales y mensuales, el promedio de cada de este.
  - **RF-8.2. Visualización de usuarios:** El administrador podrá visualizar una lista con todos los usuarios con información de cada uno.
  - **RF-8.3. Visualización de hábitos:**
    - **RF-8.3.1. Visualización de hábitos diarios:** El administrador podrá visualizar los hábitos diarios creados.
    - **RF-8.3.2. Visualización de hábitos semanales:** El administrador podrá visualizar los hábitos semanales creados.
    - **RF-8.3.3. Visualización de hábitos mensuales:** El administrador podrá visualizar los hábitos mensuales creados.

### 3.2 Requisitos no funcionales

Los requisitos no funcionales hacen referencia a características de calidad del sistema. En este proyecto, se han considerado los siguientes:

- **RNF-REND-1.1(Rendimiento):** El tiempo de carga de las vistas y respuestas del servidor debe ser de máximo 10 segundos, sin importar el número de usuarios simultáneos.
- **RNF-SEG-1.2(Seguridad):** El sistema deberá almacenar las contraseñas cifradas
- **RNF-SEG-1.3(Seguridad):** El sistema deberá debe proteger rutas privadas mediante tokens.
- **RNF-DISP-1.4(Disponibilidad):** El sistema deberá tener una disponibilidad del 99%.

- **RNF-USA-1.5(Usabilidad):** Un usuario convencional deberá ser capaz de aprender a utilizar la aplicación en menos de 1 hora.
- **RNF-USA-1.6(Usabilidad):** Un usuario deberá ser capaz de utilizar el sistema a través de un ordenador con conexión a internet.
- **RNF-INT-1.7(Interfaz):** El usuario deberá ser capaz de distinguir entre los diferentes tipos de hábitos del sistema con un simple vistazo.
- **RNF-CONF-1.8(Confidencialidad):** El usuario deberá de ser capaz de visualizar solo sus hábitos.
- **RNF-INT-1.9(Interfaz):** El sistema deberá mostrar el progreso de los hábitos del usuario de forma clara de forma que el usuario los comprenda a simple vista.

### 3.3 Roles de usuario

La aplicación contempla dos tipos de roles con diferentes niveles de acceso y funcionalidad:

- **Usuario (User):** este es el rol por defecto. Puede gestionar únicamente sus propios hábitos, ver su progreso y editar su cuenta. No tiene acceso a datos de otros usuarios ni a funciones administrativas.
- **Administrador (Admin):** este rol tiene permisos especiales para acceder al listado completo de usuarios, supervisar el uso de la aplicación y, si se requiere, gestionar usuarios o intervenir en caso de mal uso. Este tipo de cuenta está restringida y pensada para fines de mantenimiento y control.

### 3.4 Casos de uso

A continuación, se describen los principales casos de uso de la aplicación, que cubren los flujos esenciales:

### 3.4.1 Casos de uso del usuario

#### **CU1. Registrarse en la aplicación**

El usuario crea una cuenta proporcionando un correo electrónico y contraseña.

#### **CU2. Iniciar sesión**

El usuario accede a su cuenta introduciendo sus credenciales.

#### **CU3. Cerrar sesión**

El usuario finaliza su sesión de forma segura haciendo clic en “Cerrar sesión”.

#### **CU4. Cambiar contraseña**

El usuario modifica su contraseña desde el perfil, ingresando la antigua y la nueva.

#### **CU5. Eliminar cuenta**

El usuario elimina permanentemente su cuenta desde el perfil.

#### **CU6. Crear hábitos diarios, semanales o mensuales**

El usuario registra nuevos hábitos indicando nombre, frecuencia y cantidad.

#### **CU7. Modificar hábitos**

El usuario edita un hábito ya existente de cualquier tipo (diario, semanal o mensual).

#### **CU8. Eliminar hábitos**

El usuario elimina un hábito previamente creado.

#### **CU9. Marcar hábito como hecho**

El usuario indica que ha cumplido un hábito.

#### **CU10. Visualizar progreso de hábitos diarios**

El usuario ve su progreso filtrado por semana, quincena o mes, con porcentajes.

**CU11. Visualizar progreso de hábitos semanales**

El usuario accede a gráficos representativos de hábitos semanales.

**CU12. Visualizar progreso de hábitos mensuales**

El usuario consulta el resumen de los últimos 5 meses de hábitos mensuales.

**CU13. Visualizar logros**

El usuario observa un resumen con la cantidad total de hábitos completados por tipo.

**CU14. Ver perfil de usuario**

El usuario accede a su información personal.

### 3.4.2 Casos de uso del Administrador

**CU15. Acceder al panel de administración**

El administrador accede a una sección exclusiva para su rol.

**CU16. Visualizar estadísticas generales**

El administrador consulta métricas como número de usuarios y promedio de hábitos por tipo.

**CU17. Ver lista de usuarios**

El administrador accede a un listado de todos los usuarios registrados.

**CU18. Ver hábitos creados (diarios, semanales y mensuales)**

El administrador puede revisar todos los hábitos creados por los usuarios, segmentados por tipo.

## 4 Diseño del sistema

Este capítulo describe la arquitectura general del sistema, el diseño de la base de datos y la estructura del frontend y backend. El objetivo es mostrar cómo se organiza internamente la aplicación para cumplir con los requisitos especificados.

### 4.1 Arquitectura general

La aplicación sigue una arquitectura cliente-servidor, dividida en tres capas principales:

- **1. Cliente (Frontend – Angular)**

Responsable de la interacción con el usuario, visualización de la interfaz y comunicación con el backend. Se trata de una aplicación construida con Angular, que realiza peticiones HTTP para obtener y enviar datos. Todas las peticiones protegidas llevan un token JWT en la cabecera Authorization, gestionado automáticamente mediante un interceptor HTTP.

- **2. Servidor (Backend- Node.js + Express)**

Este módulo actúa como una API REST que procesa las peticiones del cliente. Se encarga de:

- Autenticación y autorización de usuarios con JWT.
- Lógica de negocio relacionada con la gestión de hábitos.
- Control de acceso según roles (usuario y administrador).
- Comunicación con la base de datos mediante consultas SQL.

Las rutas están organizadas modularmente y protegidas mediante middlewares para validación del token JWT y otro que restringe el acceso solo para administradores. Las peticiones entre frontend y backend se hacen mediante peticiones HTTP con datos en formato JSON



- **3. Almacenamiento (Base de datos – PostgreSQL)**

Se utiliza una base de datos relacional PostgreSQL para almacenar de forma persistente:

- Datos de usuarios.
- Hábitos divididos por tipo (diarios, semanales y mensuales).
- Registros de cumplimiento de hábitos (logs).

## 4.2 Diagrama de arquitectura

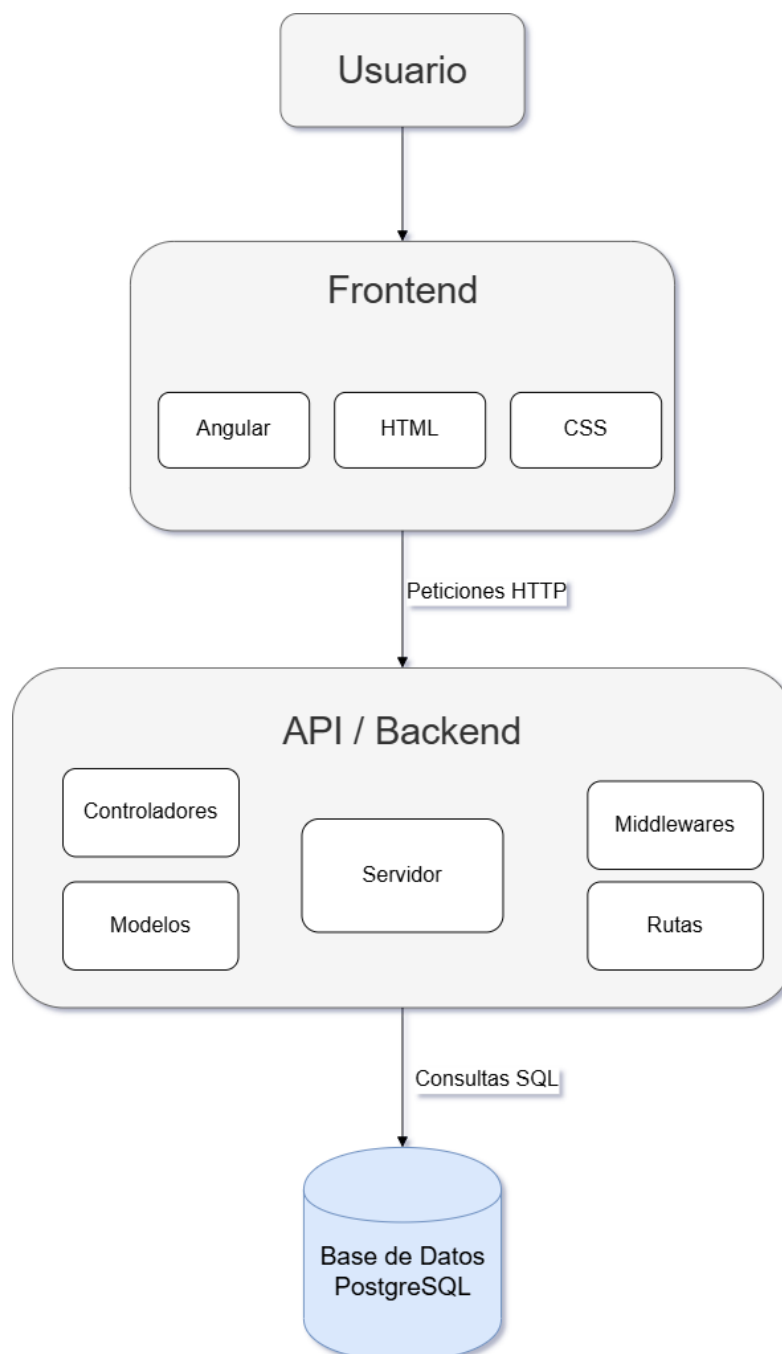


Figura 4.1 Diseño arquitectura

## 4.3 Diseño de la base de datos

La base de datos utilizada es PostgreSQL, un sistema de gestión relacional que ofrece solidez, rendimiento y soporte avanzado para integridad referencial y tipos de datos. La estructura de la base de datos ha sido diseñada para representar adecuadamente a los usuarios, sus distintos hábitos y sus registros de seguimiento.

### 4.3.1 Entidades

- **Users:** Almacena la información de los usuarios registrados
  - **Id (PK):** Identificador único de usuario.
  - **Username:** Nombre de usuario.
  - **Email:** Correo electrónico.
  - **Password:** Contraseña cifrada del usuario.
  - **Role:** Rol del usuario (user o admin).
  - **Last\_habit\_reset:** Fecha del último reinicio automático de hábitos diarios.
- **Daily\_habits:** Contiene los hábitos diarios creados.
  - **Id (PK):** Identificador único del hábito.
  - **userId (FK):** Relación con la tabla de users.
  - **Habit\_name:** Nombre del hábito.
  - **Quantity:** Cantidad esperada por día.
  - **Done:** Estado del hábito (booleano).
  - **Created\_at:** Fecha de creación.
- **Weekly\_habits:** Hábitos con frecuencia semanal (estructura similar a daily habits)
  - **id (PK)**
  - **userid (FK)**
  - **habit\_name**
  - **quantity**
  - **done**

- **created\_at**
- **Monthly\_habits:** Hábitos con frecuencia mensual.
  - **id (PK)**
  - **userid (FK)**
  - **habit\_name**
  - **quantity**
  - **done**
  - **created\_at**
- **Daily\_logs :** Registra cuándo se completa un hábito diario.
  - **habit\_id (FK)**
  - **user\_id (FK)**
  - **completed\_date (fecha del cumplimiento)**
- **Weekly\_logs y Monthly\_logs:** Estructura similar a daily\_logs, pero para registros semanales y mensuales respectivamente.

### 4.3.2 Diagrama de la base de datos

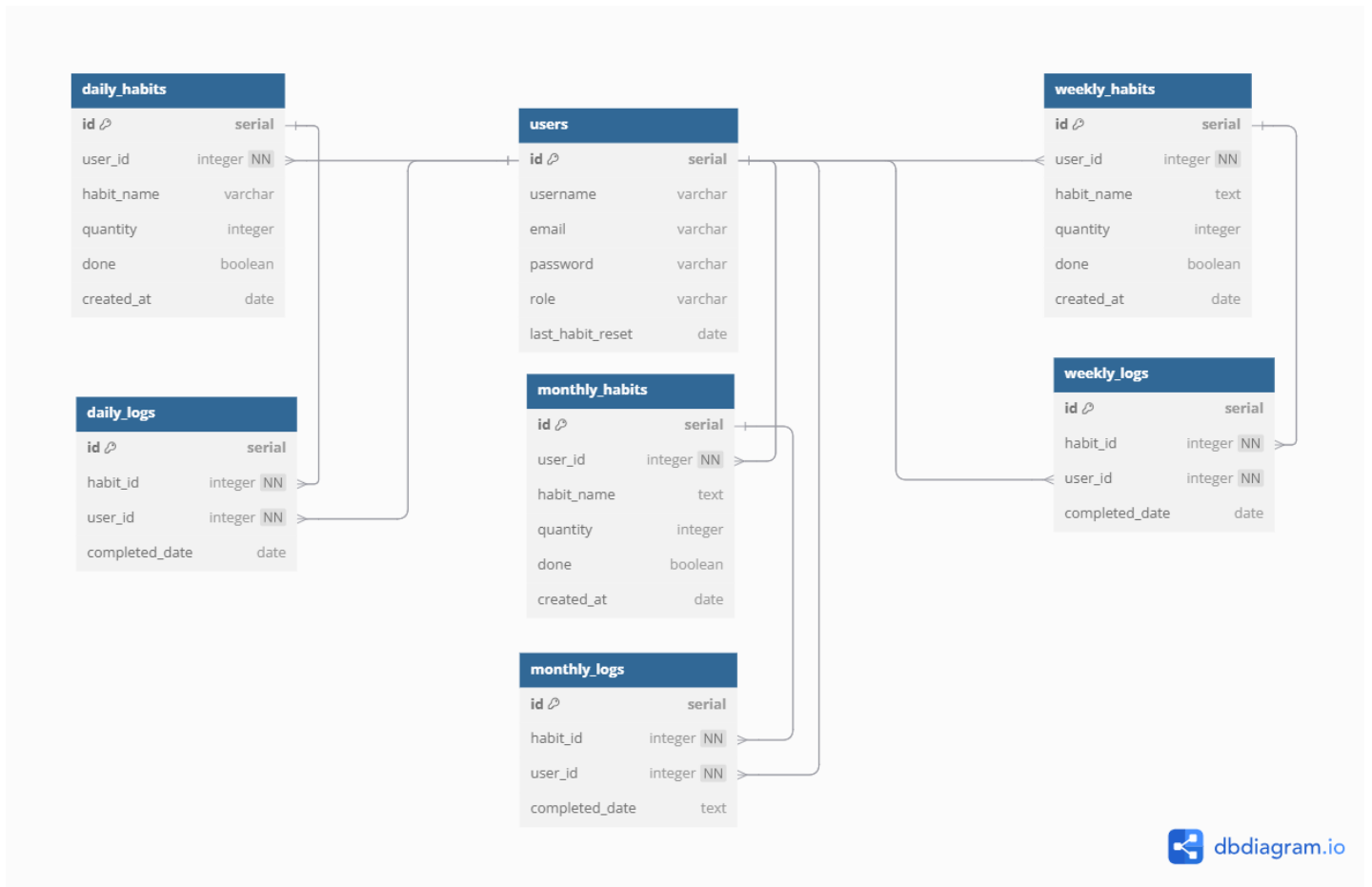


Figura 4.2 Diseño de la base de datos

El diagrama de la base de datos fue creado utilizando la herramienta [dbdiagram.io](https://dbdiagram.io).

### 4.4 Diseño del backend

El backend de la aplicación se ha desarrollado utilizando Node.js y el framework Express, siguiendo una arquitectura modular y estructurada. Su función principal es funcionar como una API REST que recibe peticiones del cliente, procesa la lógica de negocio, accede a la base de datos y devuelve las respuestas en formato JSON.

El backend está organizado en distintas fases:

#### 4.4.1 Rutas

Cada conjunto de funcionalidades está organizado en archivos de rutas independientes:

- **Auth.routes.js**

Este archivo define las rutas relacionadas con el registro, inicio de sesión y eliminación de cuentas de usuario. Las rutas están agrupadas bajo /api/auth.

- **POST /register**

Permite a un nuevo usuario registrarse enviando sus datos (nombre, correo y contraseña). Esta ruta llama al método register del controlador de autenticación.

- **POST /login**

Permite a un usuario iniciar sesión enviando sus credenciales. La ruta se conecta al método login del controlador.

- **DELETE /delete/:userId**

Elimina la cuenta de un usuario específico, identificado por su userId en la URL. Llama al método delete del controlador, enviándole el token para asegurarse de que el usuario es el que elimina el usuario.

- **User.routes.js**

Este archivo define las rutas relacionadas con la gestión del perfil, hábitos diarios y otras acciones del usuario autenticado. Todas las rutas están protegidas mediante el middleware verifyToken, lo que garantiza que solo usuarios con sesión activa puedan acceder.

A continuación, se describen las rutas principales:

- **GET /profile/:userId**

Devuelve los datos del usuario (username, email, rol) y todos sus hábitos diarios, semanales y mensuales. Devuelve un objeto con tres listas (dailyHabits, weeklyHabits, monthlyHabits).

- **GET /daily-habits/:userId**

Devuelve los hábitos diarios del usuario. Si es un nuevo día, reinicia los hábitos automáticamente y pone completed = false.

- **PATCH /updateDone/daily/:habitId**

Marca un hábito diario como completado(done=true) o no completado(done=false).

- **PATCH /updateHabit/daily/:habitId**

Permite modificar el nombre y cantidad de un habito diario.

- **POST /daily-habits**

Crea un nuevo habito diario requiriendo de userId, name y quantity.

- **DELETE /daily-habits/:dailyId**

Elimina un habito diario por su id.

- **POST /change-password**

Permite al usuario cambiar la contraseña actual verificando la contraseña anterior antes de actualizarla

- **Weekly.routes.js**

Este archivo define las rutas relacionadas con la gestión de hábitos de frecuencia semanal, así como el registro de su cumplimiento en forma de logs. Todas las rutas están protegidas mediante el middleware verifyToken, lo que garantiza que solo usuarios autenticados puedan acceder.

### **GESTIÓN DE HABITOS SEMANALES**

- **GET /weekly-habits/:userId**

Recupera todos los hábitos semanales registrados de un usuario específico.

- **POST /weekly-habits**

Crea un nuevo hábito semanal con nombre, cantidad y fecha de creación.

- **PATCH /updateDone/:habitId**

Actualiza el estado de completado(done) de un hábito semanal específico.

- **PATCH /updateHabit/weekly/:habitId**

Permite modificar los campos de los hábitos semanales (name y quantity).

- **DELETE /weekly-habits/:habitId**

Elimina un hábito semanal por su id.

## **GESTIÓN DE LOGS SEMANALES**

- **GET /weekly-logs/:userId**

Devuelve los hábitos semanales completados de un usuario en las últimas 4 semanas, agrupados por semanas.

- **POST /weekly-logs**

Registra un nuevo log indicando que un hábito ha sido cumplido en una fecha concreta.

- **DELETE /weekly-logs**

Elimina un log de un hábito semanal recibiendo los parámetros de habitId, userId y completedDate.

- **GET /weekly-logs/completed-count/:userId**

Devuelve el número de hábitos semanales completados por semana, útil para gráficas y estadísticas.

- **Monthly.routes.js**

Este módulo define las rutas encargadas de gestionar tanto los hábitos de frecuencia mensual como los registros de cumplimiento de los mismos. Todas las rutas están protegidas con verifyToken, garantizando que solo usuarios autenticados puedan acceder a ellas.

### **GESTIÓN DE HÁBITOS MENSUALES**

- **POST /monthly-habits**

Crea un nuevo habito mensual con nombre, cantidad y estado inicial done = false.

- **GET /monthly-habits/:userId**

Recupera todos los hábitos mensuales creados por un usuario específico.

- **PATCH /monthly-habits/:id**

Modifica el nombre y/o la cantidad de un habito mensual creado.

- **DELETE /monthly-habits/:id**

Elimina un hábito mensual mediante su id.

### **GESTIÓN DE LOGS MENSUALES**

- **POST /monthly-logs**

Registra el cumplimiento de un hábito mensual en una fecha concreta.

- **DELETE /monthly-logs**

Elimina el log completado de un hábito mensual por la fecha, userId y habitId

- **GET /monthly-logs/:userId**

Devuelve todos los registros de hábitos mensuales completados por un usuario



- **GET /monthly-logs/history/:userId**

Devuelve los registros de hábitos completados de un usuario de los últimos 6 meses, filtrando por fecha.

- **Habitlog.routes.js**

Este archivo define las rutas dedicadas al registro y recuperación de logs diarios (hábitos completados) por parte de los usuarios. Estas rutas permiten marcar hábitos como hechos, obtener el historial y eliminar registros de fechas concretas. Todas están protegidas mediante el middleware verifyToken.

## **GESTIÓN DE LOGS DIARIOS**

- **GET /daily-logs/:userId?range=30**

Devuelve los hábitos diarios completados por un usuario en un rango de días determinado (por defecto, 30 días). Es útil para construir tablas o gráficas de seguimiento.

- **POST /daily-logs**

Registra el cumplimiento de un hábito diario. Recibe el habitId, userId y completedDate en el cuerpo de la petición.

- **DELETE /daily-logs**

Elimina un registro de cumplimiento diario. Requiere habitId, userId y completedDate en el cuerpo para identificar el log exacto a eliminar.

- **Stats.routes.js**

Este archivo contiene rutas orientadas a proporcionar estadísticas de cumplimiento de hábitos para usuarios autenticados. Permite obtener datos agregados por mes. Está protegido mediante el middleware `verifyToken`.

- **GET /completed-logs/:userId**

Devuelve el número de hábitos completados por un usuario en cada mes, diferenciados por tipo: diarios, semanales y mensuales.

Esta ruta consulta las tablas `daily_logs`, `weekly_logs` y `monthly_logs`, agrupa los resultados por mes (YYYY-MM) de la siguiente forma:

```
{
  labels: ["2024-11", "2024-12", "2025-01", ...],
  daily: [3, 5, 7, ...],
  weekly: [2, 3, 4, ...],
  monthly: [1, 2, 2, ...]
}
```

- **Admin.routes.js**

Este archivo define las rutas exclusivas para los usuarios con rol admin. Permite acceder a funcionalidades de consulta global, como la visualización de usuarios, hábitos registrados en el sistema y estadísticas generales. Todas las rutas están protegidas mediante los middlewares `verifyToken` y `verifyAdmin`.

- **GET /stats**

Devuelve las siguientes estadísticas globales del sistema:

Usuarios registrados, hábitos diarios, hábitos semanales, hábitos mensuales, promedio diarios por usuario, promedio semanales por usuario, promedio mensuales por usuario.

- **GET /users**

Devuelve un listado de todos los usuarios registrados en la base de datos, mostrando id, username, email y role.

Cada archivo usa `express.Router()` para definir rutas.

#### 4.4.2 Middlewares

En el backend hará uso de middlewares para asegurar la seguridad de la aplicación. En esta aplicación, los middlewares se utilizan principalmente con fines de seguridad y control de acceso, permitiendo:

- Verificar si un usuario está autenticado mediante token JWT antes de ejecutar la lógica de una ruta.
- Limitar el acceso a determinadas funcionalidades en función del rol del usuario (por ejemplo, admin).
- Estandarizar comportamientos comunes sin repetir lógica en cada ruta.
- Para la implantación de la verificación en el sistema se han implementado dos tipos de middlewares:

- **VerifyToken**

Este middleware tiene como objetivo verificar que el usuario esté autenticado antes de permitirle acceder a rutas protegidas.

Extrae el token JWT del encabezado, después verifica la validez del token utilizando la clave secreta almacenada.

Si el token es válido, añade los datos del usuario (id, username, role) al objeto `req.user`, permitiendo su uso posterior en controladores o rutas. Si el token no está presente o es inválido, devuelve un error 401 o 403.

- **VerifyAdmin**

Este middleware se utiliza para restringir el acceso a rutas solo para usuarios con rol admin. Se aplica después de `verifyToken`. Estos se aplican globalmente a las rutas para garantizar que se aplica correctamente.

Requiere que req.user ya esté definido (es decir, que el token haya sido validado previamente), después verifica que el campo role en el token sea igual a 'admin'. Si no es admin, responde con un error 403 Acceso denegado.

#### 4.4.3 Auth.controller.js – Controlador

Este archivo contiene las funciones que gestionan el registro de usuarios, el inicio de sesión y la eliminación de cuentas. Está conectado a las rutas definidas en auth.routes.js.

- **Register**

Registra un nuevo usuario en la base de datos.

Recibe username, email y password desde el body de la petición, después verifica si el email ya está registrado. Cifra la contraseña usando bcryptjs y inserta el nuevo usuario en la tabla users y devuelve una respuesta de éxito.

- **Login**

Valida las credenciales del usuario y, si son correctas, genera un token JWT.

Recibe email y password desde el body, después busca el usuario por email. Compara la contraseña proporcionada con el hash guardado, si la autenticación es válida, entonces, se genera un JWT con id, username y role y lo devuelve como respuesta.

- **Delete**

Elimina la cuenta de un usuario, junto con sus hábitos y registros asociados.

Verifica que el userId del token coincida con el del usuario que desea eliminar, o que sea un administrador, después borra los registros (daily\_logs, weekly\_logs, etc.) relacionados con ese usuario y elimina los hábitos y finalmente, elimina al usuario de la tabla users. Esta protegido

por `verifyToken` que solo se permite eliminar la cuenta propia o, si se es admin, cualquier cuenta.

#### 4.4.4 Modelo de datos

El backend utiliza consultas SQL directas sobre un pool de PostgreSQL.

El model de `user.model` abstrae las operaciones como crear, buscar o eliminar usuarios. A diferencia del uso de un ORM (Object Relational Mapping), se ha optado por ejecutar consultas SQL directas, lo que otorga mayor control sobre la estructura y el rendimiento de las operaciones.

#### 4.4.5 Seguridad

La seguridad ha sido una prioridad fundamental en el desarrollo de la aplicación, tanto en la protección de los datos personales de los usuarios como en el control de acceso a los distintos recursos del sistema. Se han implementado mecanismos estándar del desarrollo web moderno, incluyendo cifrado de contraseñas, autenticación mediante JWT y middleware de autorización por roles.

- **Cifrado de contraseñas**

Todas las contraseñas de los usuarios se almacenan de forma segura utilizando la librería `bcryptjs`, que aplica un proceso de hashing unidireccional. Este método garantiza que incluso si la base de datos fuera comprometida, las contraseñas no puedan ser recuperadas en texto plano.

Al registrarse, la contraseña es cifrada antes de ser guardada en la base de datos.

Durante el login, se utiliza `bcrypt.compare()` para verificar si la contraseña ingresada coincide con el hash almacenado.

- **Autenticación con JWT**

Al iniciar sesión correctamente, el backend genera un token JWT (JSON Web Token) que incluye información codificada del usuario: id, username, role.

Este token es firmado con una clave secreta (JWT\_SECRET) y tiene un tiempo de expiración predefinido (en este caso se uso 1 hora). El token se envía al cliente, que lo almacena en localStorage y lo incluye en todas las peticiones protegidas mediante el encabezado:

## 4.5 Diseño del frontend

El frontend de la aplicación se ha desarrollado utilizando el framework Angular, una de las tecnologías más utilizadas en el desarrollo de aplicaciones web modernas gracias a su arquitectura modular, sistema de componentes y herramientas integradas para manejo de rutas, servicios y formularios reactivos.

La interfaz de usuario ha sido diseñada para ser clara, intuitiva y orientada a la usabilidad, facilitando el registro, seguimiento y análisis de hábitos personales.

### 4.5.1 Estructura general del frontend

La aplicación Angular se estructura mediante **componentes independientes y especializados**, organizados por funcionalidades. Esta separación permite mantener el código limpio, modular y escalable.

## Componentes

Cada vista o sección funcional se representa mediante un componente.

- **Login**

Permite al usuario iniciar sesión introduciendo su email y contraseña. Al validar las credenciales correctamente, se almacena el token JWT en el localStorage del navegador, lo que permite realizar peticiones autenticadas al backend.

El componente también maneja errores como credenciales inválidas y redirige al usuario a /home si el inicio de sesión es exitoso.

La vista se compone de un formulario reactivo con validación de campos, mensajes de error y estilo coherente con el resto de la aplicación.

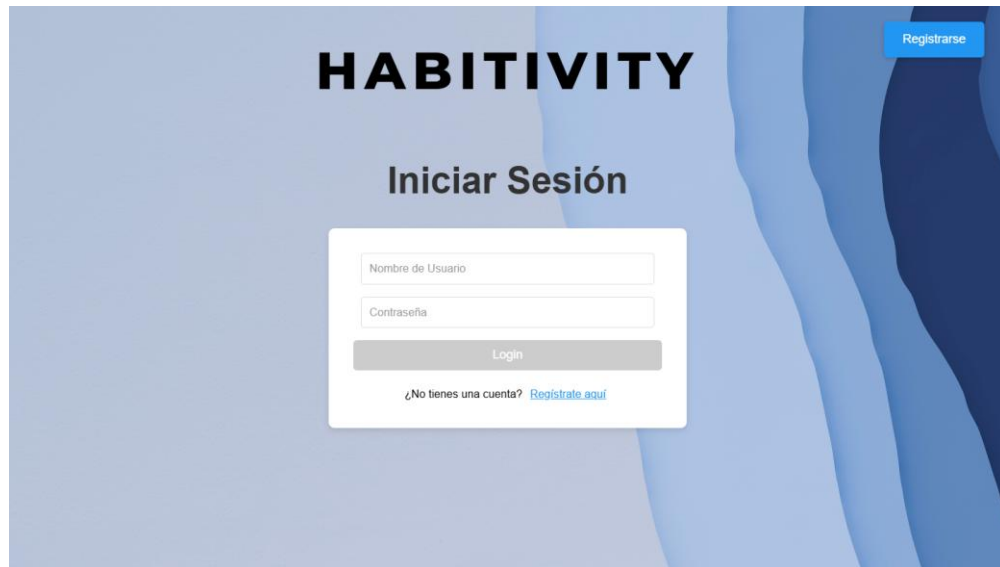


Figura 4.3 Pagina de inicio de sesión

- **Register**

Permite a nuevos usuarios crear una cuenta. Se validan campos como nombre de usuario, email y contraseña, y se gestiona la respuesta del servidor para mostrar errores si el usuario ya existe. Si el registro es exitoso, redirige al usuario a la página de login.

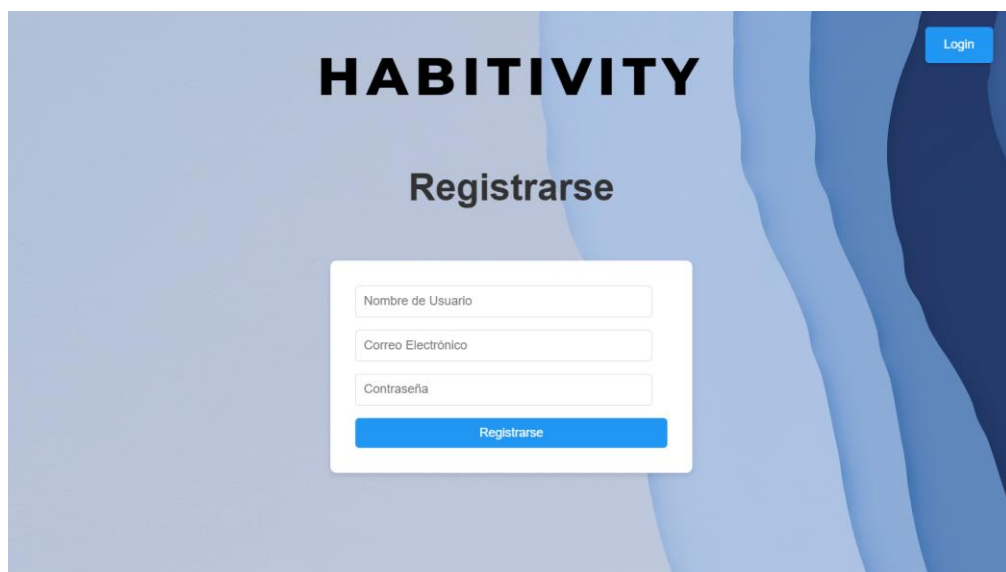


Figura 4.4 Página de registro

- **Home**

La página principal HomeComponent es la parte más importante de la experiencia del usuario dentro de la aplicación. Desde aquí, los usuarios pueden visualizar y gestionar sus hábitos diarios, semanales y mensuales, así como acceder a estadísticas sobre su progreso.

**Funcionalidades:**

- **Carga automática de hábitos:** Al iniciar la aplicación, se cargan automáticamente los hábitos del usuario autenticado a través de peticiones HTTP (GET) a las rutas protegidas del backend. Esto incluye:
  - **/api/user/daily-habits/:userId**
  - **/api/weekly/weekly-habits/:userId**
  - **/api/monthly/monthly-habits/:userId**
- **Visualización dinámica de hábitos**

Los hábitos se presentan organizados. El usuario puede ver su estado (completado o no) y su cantidad establecida. Se utilizan componentes visuales para mostrar esta información de forma clara y comprensible.
- **Gestión de hábitos**

Desde esta vista, el usuario puede modificar, marcar como realizados o eliminar sus hábitos a través de botones e interfaces intuitivas que disparan métodos como:

  - **updateDoneHabit()** : Para actualizar el estado de completado del hábito.
  - **editHabit()**: Para modificar los atributos de nombre y cantidad del hábito.
  - **deleteHabit()**: Para eliminar un hábito.



- **Gráficas de progreso**

El HomeComponent integra varias secciones, para cada tipo de hábito, para representar el progreso de los hábitos. Cada tipo de hábito se distingue por tener un tipo de gráfica distinta.

- **Daily habits**

Para representar el progreso del usuario en relación con sus hábitos diarios, se han implementado dos tipos de gráficas que permiten una visualización intuitiva y clara de los datos:

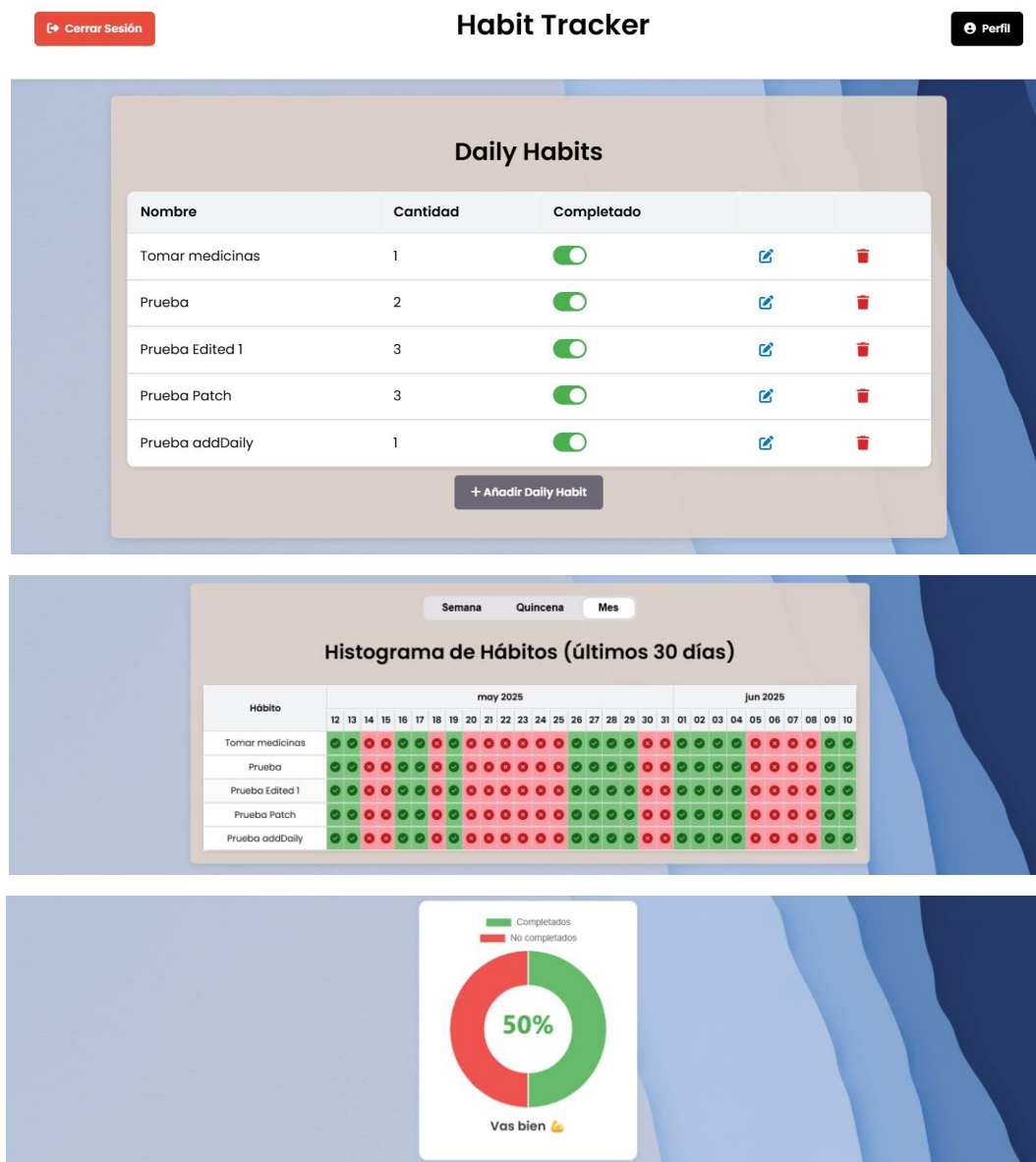
- **Histograma de cumplimiento diario:**

Esta gráfica muestra en formato de matriz los hábitos registrados. Cada fila representa un hábito diario y cada columna un día del mes. Las celdas dentro de la matriz indican si el hábito fue cumplido (V) o no (X) en cada uno de los días. Esta visualización permite al usuario identificar patrones, constancia y posibles áreas de mejora en sus hábitos diarios.

- **Gráfica tipo donut (anillo):**

Esta gráfica muestra el porcentaje de cumplimiento total de los hábitos diarios dentro de un rango de días seleccionados. Según el porcentaje alcanzado, se generan mensajes motivacionales automáticos para fomentar la constancia del usuario y reforzar conductas positivas. Esta gráfica es útil como resumen rápido del rendimiento.

Ambas gráficas representan los días seleccionados por el usuario que en este caso son 3 posibilidades planteadas. El usuario puede elegir representar las gráficas de progreso de una semana, una quincena o un mes.



Figuras 4.5-4.7 Diseño Daily Habits

#### ▪ Weekly habits

La sección de hábitos semanales en la página principal incluye una gráfica de tipo línea temporal, que representa la evolución del número de hábitos semanales completados por el usuario durante las últimas 6 semanas, además, la tabla de los hábitos semanales cuenta con un historial para cada hábito de las ultimas 5 semanas.

- **Características:**

Eje X: Muestra las fechas de inicio de cada semana.

Eje Y: Indica la cantidad total de hábitos semanales completados durante esa semana.

Puntos de datos: están coloreados para facilitar una interpretación visual rápida del rendimiento (verde-alto, naranja-medio, rojo-bajo)

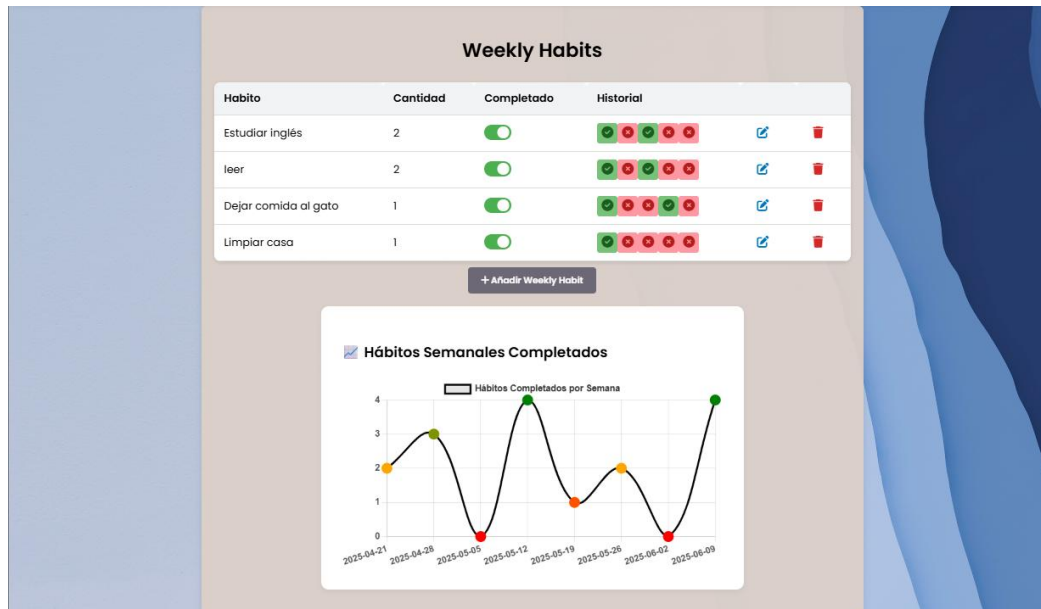


Figura 4.8 Diseño Weekly Habits

#### ▪ Monthly habits

La sección de hábitos mensuales en la página principal incluye una gráfica de tipo línea temporal, que representa la evolución del número de hábitos mensuales completados por el usuario durante los últimos 6 meses.

- **Características:**

**Eje X:** representa los últimos 6 meses en los que el usuario ha registrado actividad.

**Eje Y:** muestra la cantidad total de hábitos mensuales completados en cada mes.

**Barras:** las barras están coloreadas en diferentes tonos según el número de hábitos cumplidos, lo que permite una interpretación rápida del desempeño mensual.

Esta visualización ayuda al usuario a identificar de forma clara su evolución a lo largo del tiempo, destacar meses especialmente productivos y detectar periodos donde hubo menor constancia. El propósito es fomentar la regularidad en el cumplimiento de hábitos a largo plazo.

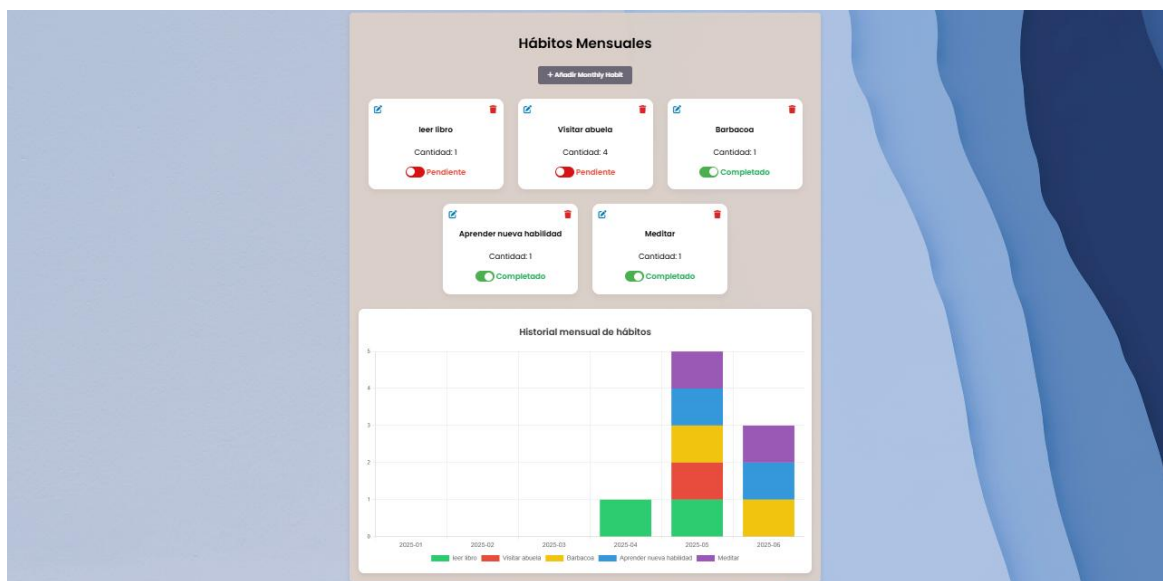


Figura 4.9 Diseño Monthly Habits

## ○ Profile

El componente ProfileComponent permite al usuario autenticado consultar información relacionada con su cuenta y sus hábitos. Es una vista personal que agrega funciones de gestión de cuenta, visualización de estadísticas y navegación.

### Funcionalidades principales:

- Visualización del perfil

Al cargar la vista, se obtiene del backend la información del usuario (nombre, correo y rol) junto con sus hábitos agrupados por tipo (diarios, semanales y mensuales). Se muestra también un resumen de hábitos completados.

- **Gráfica de progreso mensual**

La sección incluye una gráfica lineal que representa la evolución de los hábitos completados por tipo (daily, weekly y monthly) durante los últimos seis meses. Cada línea corresponde a un tipo de hábito y su progreso mensual, lo que permite al usuario detectar tendencias, mejoras o retrocesos en su constancia.

- **Resumen de logros**

Justo debajo de la gráfica, se muestran contadores del número total de hábitos completados por tipo. Esta métrica refuerza la motivación del usuario al mostrar su progreso acumulado.

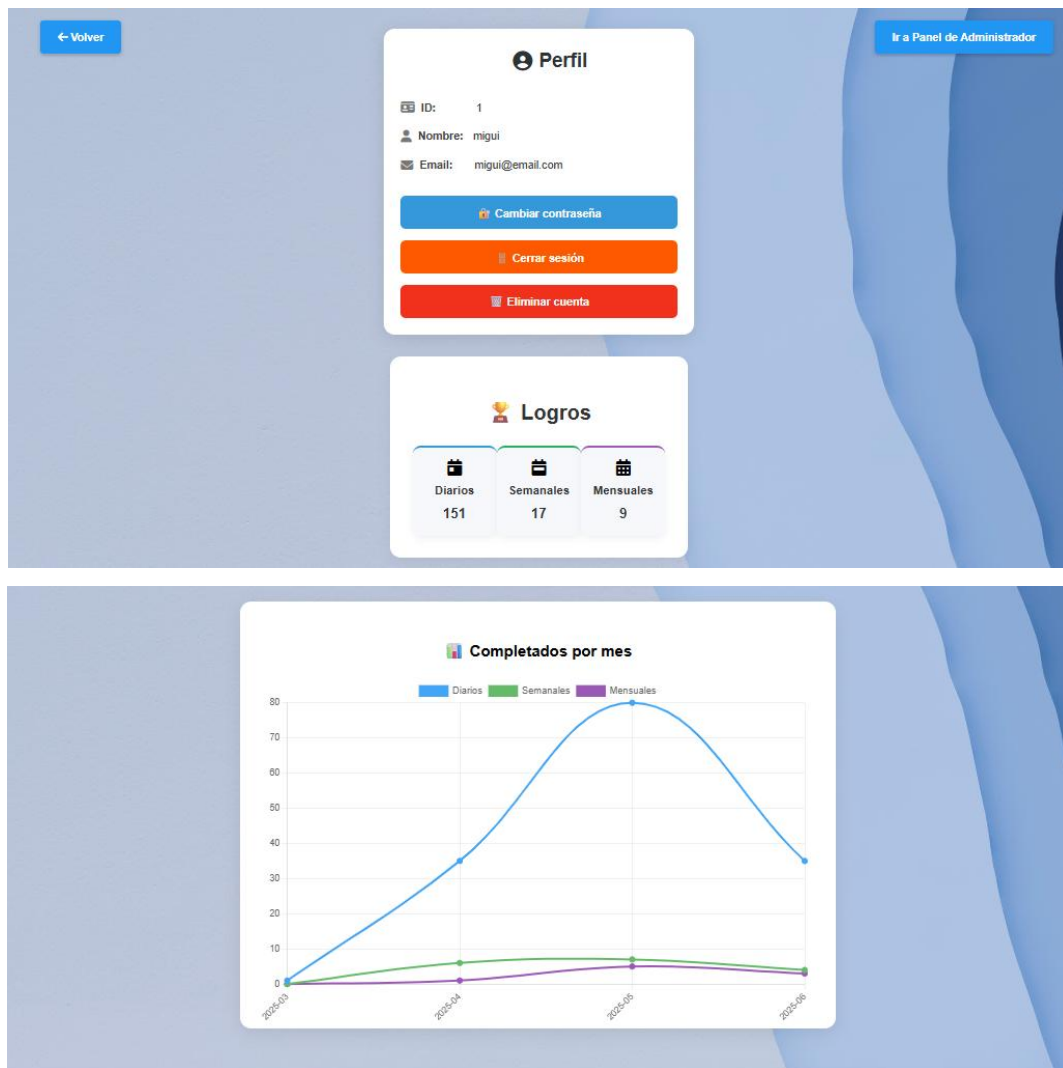
- **Gestión de cuenta**

Desde esta vista, el usuario puede:

- Cambiar su contraseña mediante un diálogo modal que verifica la contraseña actual.
- Eliminar su cuenta, tras confirmar en un diálogo personalizado. Esta acción elimina al usuario de la base de datos.
- Cerrar sesión, lo que limpia el token y redirige al login.

- **Acceso a panel de administrador (solo si el usuario tiene rol admin)**

Si el usuario autenticado es un administrador, se le ofrece un botón para redirigir al panel de administración, desde donde podrá ver estadísticas generales y gestión de usuarios.



Figuras 4.10 y 4.11 Diseño profile

- **Admin-dashboard:** Panel exclusivo para usuarios de rol admin

El componente AdminDashboardComponent está diseñado para ofrecer una vista exclusiva para los usuarios con rol de administrador. Desde esta sección se pueden visualizar estadísticas generales de la plataforma, así como un listado completo de usuarios registrados en el sistema. Esto permite al administrador tener una visión global del uso y comportamiento de los usuarios.

- **Estadísticas Generales**

Una vez autenticado un administrador, se realiza una consulta al backend mediante el servicio `getAdminStats`, que devuelve un conjunto de métricas claves:

- **Usuarios registrados:** Número total de usuarios en la base de datos.
- **Hábitos creados:** Desglosado por tipo (diarios, semanales y mensuales).
- **Promedio de hábitos por usuario:** Promedios de hábitos diarios, semanales y mensuales por usuario.

Estas estadísticas se muestran de forma clara en una tarjeta visual (card) en la parte superior de la página.

#### • Listado de Usuarios

Se incluye un botón interactivo que permite alternar la visibilidad de una tabla con los usuarios registrados. Al hacer clic en "Ver usuarios", se realiza una llamada al método `getAllUsers` que recupera la lista completa de usuarios, incluyendo los siguientes datos:

- **ID de usuario**
- **Nombre de usuario**
- **Correo electrónico**
- **Rol (usuario o administrador)**

Esto permite al administrador tener control y seguimiento sobre la base de usuarios, facilitando la gestión de roles o revisión de actividad si fuera necesario.

#### • Funcionalidad

Todo este comportamiento está encapsulado dentro del componente Angular, usando suscripciones a servicios asíncronos (`AuthService`). Además, se aprovecha Angular para mostrar u ocultar elementos dinámicamente según el estado de la variable `showUsers`.



Figura 4.12 Diseño Admin Dashboard

○ **Change-password-dialog y confirm-dialog**

Se han implementado dos componentes auxiliares reutilizables que mejoran la experiencia del usuario al realizar acciones sensibles:

- **ChangePasswordDialog**

Permite a los usuarios modificar su contraseña de forma segura. Requiere ingresar la contraseña actual, una nueva, y confirmar la nueva contraseña. El sistema valida que las contraseñas coincidan y que no sean iguales, y en caso de éxito, muestra un mensaje de confirmación.

- **ConfirmDialogComponent**

Muestra una ventana modal de confirmación que se utiliza antes de



realizar acciones importantes como la eliminación de una cuenta. Ofrece opciones para cancelar o confirmar la operación.

Estos componentes se integran especialmente dentro de la vista de perfil, permitiendo al usuario mantener control y seguridad sobre su cuenta.

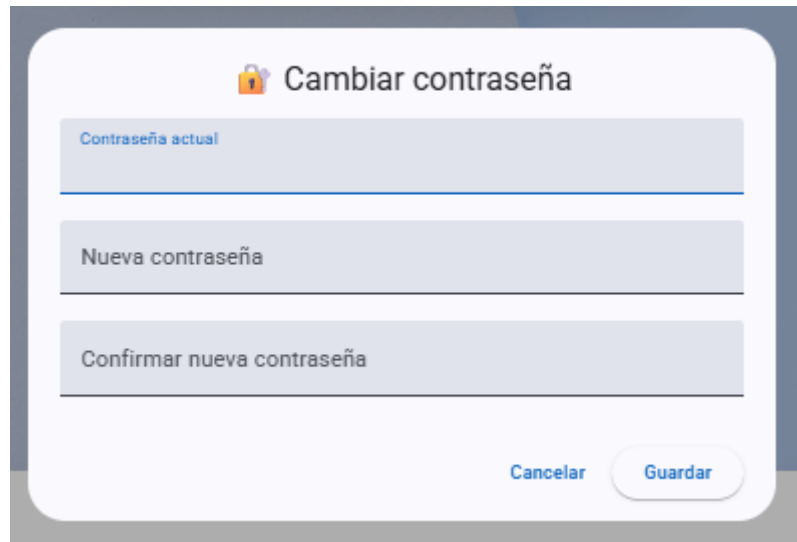
The image shows a 'Cambiar contraseña' (Change Password) dialog box. It has a title bar with a lock icon and the text 'Cambiar contraseña'. Below the title bar are three input fields: 'Contraseña actual' (Current password), 'Nueva contraseña' (New password), and 'Confirmar nueva contraseña' (Confirm new password). At the bottom right, there are two buttons: 'Cancelar' (Cancel) and 'Guardar' (Save).

Figura 4.13 Diseño ChangePasswordDialog

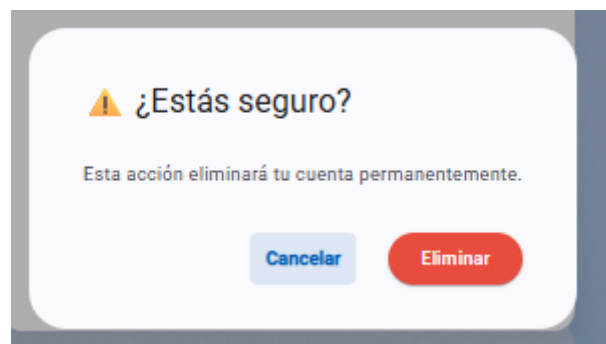
The image shows a '¿Estás seguro?' (Are you sure?) confirmation dialog box. It has a title bar with a warning icon and the text '¿Estás seguro?'. Below the title bar is a message: 'Esta acción eliminará tu cuenta permanentemente.' (This action will permanently delete your account.). At the bottom, there are two buttons: 'Cancelar' (Cancel) and 'Eliminar' (Delete).

Figura 4.14 Diseño ConfirmDialogComponent

- **Models – User.model**

En el frontend se utiliza el modelo UserWithHabits para representar un usuario junto con sus hábitos asociados, agrupados por tipo. Este modelo facilita el acceso estructurado a la información del usuario desde los componentes del perfil o administrador.

Descripción de los campos:

- **id:** Identificador único del usuario.
- **username:** Nombre de usuario utilizado para autenticación y visualización.
- **email:** Dirección de correo electrónico asociada al usuario.
- **role:** Rol asignado al usuario (por ejemplo: user o admin), usado para el control de acceso.
- **dailyHabits:** Lista de hábitos marcados como diarios, que se espera que se completen cada día.
- **weeklyHabits:** Lista de hábitos con frecuencia semanal.
- **monthlyHabits:** Lista de hábitos que deben cumplirse una vez al mes.

Este modelo permite agrupar la información relevante del usuario y sus hábitos de manera lógica y organizada, lo que facilita la generación de gráficas de progreso y otras funcionalidades en la interfaz de usuario.

- **Servicios**

En la arquitectura frontend de la aplicación, la gestión de la autenticación y la comunicación con el backend se centraliza en el servicio **auth.service.ts**. Este servicio es una pieza clave del sistema, ya que abstrae y organiza toda la lógica relacionada con la identidad del usuario y las operaciones protegidas.

Entre sus responsabilidades principales se encuentran:

- **Peticiones HTTP al backend:** Todas las llamadas a la API REST para registrar usuarios, iniciar sesión, obtener datos del perfil, estadísticas o gestionar hábitos, se realizan a través de métodos definidos en `auth.service.ts`. De esta forma, los componentes mantienen un enfoque limpio y centrado únicamente en la presentación y lógica de interfaz.
- **Gestión del token JWT:** Tras una autenticación exitosa, el token JWT devuelto por el backend se almacena localmente (en `localStorage`). El servicio se encarga de su almacenamiento, recuperación y eliminación (al cerrar sesión).

- **Decodificación del token:** El servicio extrae información importante del token, como el `userId` y el `role` (usuario o administrador), para determinar los permisos y el contenido que puede visualizar el usuario.
- **Abstracción de operaciones:** Por ejemplo, métodos como `getUserProfile()`, `getAdminStats()`, `changePassword()` o `deleteAccount()` encapsulan llamadas HTTP concretas con su correspondiente endpoint y tipo de operación (GET, POST, PUT, DELETE, etc.).

Esto hace que el `auth.service.ts` actúe como el "puente" entre el frontend y la lógica del servidor, lo que promueve un código más limpio, reutilizable y mantenible.

- **Interceptor de Autenticación**

Para asegurar que todas las peticiones hacia rutas protegidas lleven el token JWT, se implementó un interceptor HTTP (`auth.interceptor.ts`). Este interceptor intercepta todas las peticiones salientes y añade automáticamente en la cabecera `Authorization: Bearer <token>` con un token válido. Esto evita tener que gestionar manualmente las cabeceras en cada componente o servicio, garantizando así una seguridad consistente y transparente.

- **Seguridad**

Para evitar el acceso no autorizado a rutas protegidas, se ha implementado un **`guard(auth.guard.ts)`** que impide la navegación hacia rutas como `/home` o `/profile` si el usuario no ha iniciado sesión. Este guard verifica si existe un token válido en el navegador.

- **Usabilidad**

Se ha puesto especial atención en diseñar una interfaz limpia e intuitiva. Se emplean componentes visuales como botones claramente etiquetados, colores suaves, y gráficos interactivos para que los usuarios puedan entender fácilmente su progreso de hábitos. La navegación es fluida, con una estructura clara que guía al usuario desde el inicio de sesión hasta la exploración y gestión de sus hábitos, estadísticas y perfil.

## 5 Pruebas del sistema

### 5.1. Pruebas unitarias

Durante el desarrollo del frontend se han incorporado pruebas unitarias para verificar la funcionalidad básica de componentes y servicios críticos, especialmente aquellos relacionados con la autenticación y el control de acceso.

#### Servicios

- **AuthService:** Se utilizó Jasmine y Karma para generar pruebas automáticas que aseguran la correcta inicialización del servicio. Estas pruebas validan que el servicio se cree sin errores, lo cual es esencial para asegurar la disponibilidad del sistema de autenticación. Aunque las pruebas generadas son básicas, sirven como punto de partida para expandir futuros tests personalizados que cubran funcionalidades como login, logout, getToken, y getUserId.

#### Guards

- **AuthGuard:** Se realizaron validaciones para comprobar que el guardián de rutas bloquea correctamente el acceso a rutas protegidas si el usuario no ha iniciado sesión. Este componente garantiza que las rutas como /home y /profile sean accesibles solo cuando hay un token válido.

### 5.2 Pruebas funcionales

Con el objetivo de garantizar que la aplicación cumpla con los requerimientos funcionales definidos, se llevaron a cabo pruebas funcionales que simulan la interacción real del usuario con el sistema. Estas pruebas permitieron verificar que los flujos de uso más relevantes operan correctamente de extremo a extremo.

#### Casos funcionales cubiertos:

- **Registro y autenticación de usuarios:** Se verificó que el proceso de registro crea un nuevo usuario correctamente y que el login valida credenciales, almacena el token JWT y redirige a la vista principal según el rol.

- **Gestión de hábitos:** Se probó la creación, edición y eliminación de hábitos diarios, semanales y mensuales, asegurando que los cambios se reflejen en la interfaz y en el backend.
- **Seguimiento del progreso:** Se comprobó que el usuario puede marcar hábitos como completados y que dichos registros actualizan las gráficas de progreso correctamente.
- **Visualización de estadísticas:** Se validó la generación de gráficas interactivas (histograma, donut, línea) que representan el desempeño del usuario en distintos rangos temporales.
- **Seguridad de navegación:** Se confirmó que las rutas protegidas (como /home, /profile o /admin-dashboard) son accesibles solo para usuarios autenticados con el rol correspondiente.
- **Panel de administración:** Se comprobó que los usuarios con rol de administrador acceden a estadísticas globales y listado de usuarios correctamente.
- **Opciones de cuenta del usuario:** Se probaron funcionalidades de cierre de sesión, cambio de contraseña y eliminación de cuenta, validando su ejecución correcta y retroalimentación visual mediante mensajes.

Estas pruebas se realizaron manualmente durante el desarrollo, simulando los comportamientos esperados desde la perspectiva del usuario final, lo que permitió detectar errores de flujo y validar la experiencia general de uso.

### 5.3. Pruebas de API con Postman

Para garantizar que cada funcionalidad del backend estuviera correctamente implementada, se realizó una batería de pruebas manuales usando Postman, abarcando todos los endpoints definidos en la API REST. Estas pruebas verificaron tanto respuestas esperadas como el manejo de errores, autenticación y consistencia de datos.

#### Endpoints cubiertos en las pruebas:

- **User**

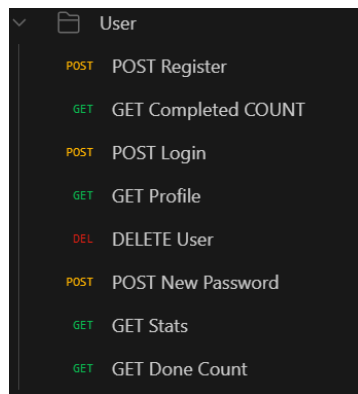


Figura 5.1 Pruebas Postman del usuario

- **Daily habits**

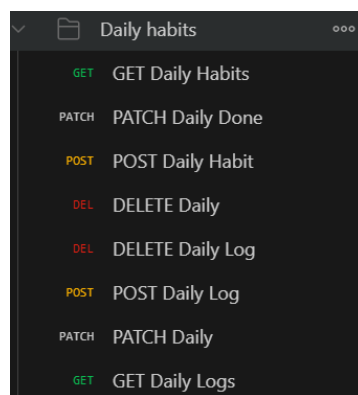


Figura 5.2 Pruebas Postman de los daily habits

- **Weekly habits**

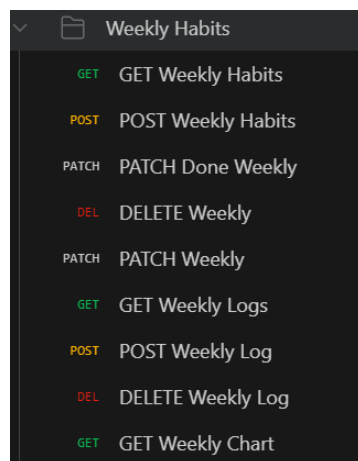


Figura 5.2 Pruebas Postman de los weekly habits

- **Monthly habits**

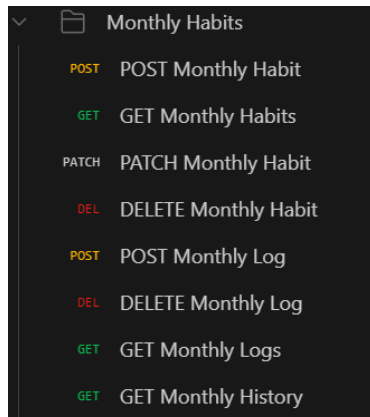


Figura 5.3 Pruebas Postman de los monthly habits

- **Admin**

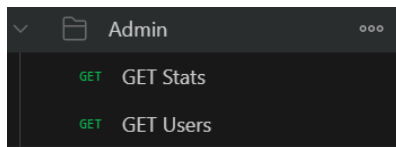


Figura 5.4 Pruebas Postman de Administrador

Cada uno de estos endpoints fue probado con distintos escenarios, como:

- Datos válidos vs inválidos
- Tokens ausentes, caducados y válidos
- Códigos de estado esperados (200 OK, 401 Unauthorized, 403 Forbidden, 400 Bad Request, etc.)

Estas pruebas confirmaron que la API responde de forma robusta ante peticiones correctas y errores, garantizando fiabilidad tanto para el frontend como para futuros clientes (por ejemplo, apps móviles).

## 6. Impacto social y medioambiental

### 6.1 Impacto social

Esta aplicación tiene un fuerte componente social, ya que está orientada al bienestar personal, la autoorganización y la disciplina. En una sociedad donde el estrés, la desorganización y la distracción digital son problemas crecientes, ofrecer una herramienta accesible para gestionar hábitos puede tener un efecto positivo en diversos ámbitos:

- **Salud mental:** Fomentar rutinas saludables ayuda a reducir la ansiedad, mejorar el descanso y aumentar la autoestima.
- **Productividad personal:** El seguimiento visual del progreso incentiva la constancia y el logro de metas diarias.
- **Desconexión digital consciente:** Promueve el uso responsable del tiempo frente a pantallas, frente al uso pasivo de redes sociales.

La solución puede ser adoptada tanto por individuos como por colectivos educativos o terapéuticos interesados en el desarrollo personal, la mejora de hábitos o el seguimiento de rutinas saludables.

## 6.2 Impacto medioambiental

Aunque el proyecto no está directamente vinculado con procesos productivos o materiales físicos, su diseño presenta algunas características relevantes desde el punto de vista medioambiental:

- **Desmaterialización:** al ser una aplicación completamente digital, no genera residuos ni requiere producción de dispositivos específicos.
- **Accesibilidad multiplataforma:** al funcionar desde cualquier navegador web, evita la necesidad de desarrollos duplicados en distintos sistemas operativos o dispositivos físicos.
- **Optimización del código y eficiencia energética:** El uso de tecnologías modernas y ligeras como Angular y Node.js permite una ejecución eficiente, con un consumo reducido.

En conjunto, se trata de una herramienta tecnológica respetuosa con el entorno, con potencial para escalar sin aumentar su huella ambiental.



## 7. Conclusiones

El desarrollo de esta aplicación web para la gestión de hábitos ha permitido demostrar la viabilidad de crear una herramienta accesible, visual y funcional, centrada en el acompañamiento del usuario en su camino hacia la consolidación de rutinas saludables y productivas. A lo largo del proyecto, se han alcanzado los objetivos principales: permitir la creación, seguimiento y análisis de hábitos con distintas frecuencias (diaria, semanal y mensual), así como facilitar al usuario una experiencia clara e intuitiva desde cualquier dispositivo con navegador.

A nivel técnico, el uso de tecnologías modernas como Angular, Node.js y PostgreSQL ha demostrado ser adecuado para construir una aplicación robusta y escalable. La implementación de gráficos interactivos, autenticación mediante JWT, y la gestión diferenciada por roles (usuario y administrador) ha enriquecido notablemente la funcionalidad de la plataforma.

Desde el punto de vista personal, este proyecto ha supuesto una oportunidad para profundizar en el desarrollo full stack, afianzar conocimientos de arquitectura web y enfrentarse a problemas reales de diseño, rendimiento y seguridad.

Por último, cabe destacar que el proyecto sienta las bases para futuras ampliaciones, como la inclusión de notificaciones, una versión móvil nativa o una comunidad de usuarios. La herramienta desarrollada no solo cumple con los requisitos funcionales planteados inicialmente, sino que aporta un valor tangible en el ámbito del desarrollo personal y la mejora continua.

### 7.1 Líneas futuras de mejora

La aplicación desarrollada cumple con los objetivos funcionales establecidos, existen múltiples posibilidades de mejora y ampliación que podrían abordarse en futuros desarrollos:

- **Desarrollo de aplicación móvil nativa**

Aunque actualmente la aplicación es accesible desde cualquier navegador, crear versiones específicas para Android y iOS permitiría ofrecer notificaciones push, mayor rendimiento en dispositivos móviles y una experiencia de usuario más optimizada. además de ofrecer más comodidad al usuario común que accede desde el teléfono móvil y dando mas visibilidad a la aplicación.

- **Sistema de recordatorios y notificaciones**

Añadir recordatorios programados ayudaría a mejorar la constancia del usuario en la realización de sus hábitos, especialmente en hábitos con frecuencia semanal o mensual.

- **Sistema de recompensas o logros**

Integrar elementos como logros, insignias o recompensas visuales cuando el usuario alcance unos objetivos que podrían motivar a los usuarios sin comprometer la simplicidad del sistema. Esta funcionalidad debería ser opcional para mantener la interfaz limpia.

- **Interacción social**

Permitir compartir logros con otros usuarios como amigos o poder visualizar los logros de personas que el usuario admire(idolos o figuras públicas) que podrían servir como fuente de motivación e inspiración, fomentando la motivación y el compromiso personal a través del ejemplo.

- **Más tipos de visualización del progreso**

Incorporar nuevos tipos de gráficas (por ejemplo, barras comparativas entre semanas o mapas de calor) que podrían enriquecer el análisis del rendimiento personal.

- **Gestión avanzada de hábitos**

Ofrecer parámetros adicionales como días específicos de la semana, duración estimada o prioridad permitiría una personalización aún mayor de los hábitos.

- **Internacionalización**

Traducir la interfaz a varios idiomas ampliaría el alcance de la aplicación a usuarios de diferentes regiones.

- **Inclusión de mensajes motivacionales**

Aumentar la cantidad y variedad de los mensajes motivadores de la aplicación basados en el rendimiento del usuario podría aportar al usuario un reforzamiento en la motivación del usuario.

- **Inclusión de contenido motivacional de expertos**

Añadir cápsulas de vídeo breves con consejos de psicólogos, entrenadores o expertos en productividad aportaría valor añadido y rigor para que el usuario pueda aprender a establecer una rutina a partir del conocimiento de personas expertas.

- **Sugerencias de hábitos por nivel**

Incluir un sistema de recomendación que sugiera hábitos organizados por niveles (principiante, intermedio, avanzado) ayudaría a guiar al usuario en función de su experiencia.

Estas posibles líneas de mejora no solo ampliarían las funcionalidades del sistema, sino que también podrían adaptarlo a una base de usuarios más amplia, incrementando su valor como herramienta de apoyo al desarrollo personal.

## Bibliografía

- Angular. (s.f.). *Angular - The modern web developer's platform*. <https://angular.io>
- Auth0. (s.f.). *Introduction to JSON Web Tokens*. <https://auth0.com/learn/json-web-tokens>
- Fabulous. (s.f.). *Fabulous App*. <https://www.thefabulous.co>
- Google. (s.f.). *Google Tasks*. <https://www.google.com/tasks>
- HabitBull. (s.f.). *HabitBull Habit Tracker*. <https://www.habitbull.com>
- Habitica. (s.f.). *Habitica - Gamify your life*. <https://habitica.com>
- Habitify. (s.f.). *Habitify - Habit Tracker*. <https://habitify.me>
- Mozilla Developer Network. (s.f.). *JavaScript Reference*. <https://developer.mozilla.org>
- Node.js. (s.f.). *Node.js Documentation*. <https://nodejs.org/en/docs>
- pgAdmin Team. (s.f.). *pgAdmin - PostgreSQL Tools*. <https://www.pgadmin.org>
- PostgreSQL Global Development Group. (s.f.). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- Stack Overflow. (s.f.). *Preguntas y respuestas de la comunidad*. <https://stackoverflow.com>
- Visual Studio Code. (s.f.). *Visual Studio Code Documentation*. <https://code.visualstudio.com/docs>
- W3Schools. (s.f.). *HTML, CSS and JavaScript Tutorials*. <https://www.w3schools.com>
- nodemon. (s.f.). *Nodemon - Monitor for changes in your node.js application*. <https://www.npmjs.com/package/nodemon>

- dbdiagram. Para la creación del diagrama de la base de datos.

<https://dbdiagram.io/>